

SECURITY-AUDIT BUGS ARE FAILED DISTRIBUTIVE LAWS A GRANT-IMPACT CASE STUDY FOR COMPOSITIONAL CORRECTNESS

MACBETH

ABSTRACT. Real-world smart-contract exploits repeatedly take the same shape: two or more components, each provably correct in isolation, produce a global failure when composed. This is precisely the picture that distributive-law theory names: composition of effect structures is coherent only when a distributive law exists, and its absence is an obstruction. We frame three recurring audit failure classes — state divergence, temporal/cross-callback re-entrancy, and self-composition via iteration — as three *different* categorical laws breaking, work a clean toy example (the Provenance Blockchain state-divergence bug) as a failed distributive law between two component monads, and connect the temporal case to an existing machine-checked obstruction result. The point is not a new theorem but a unifying frame with predictive and economic bite.

Status: expository grant-Impact note, not a peer-reviewed result. The only formally verified claim relied on here is a previously established Lean result on re-entrancy (referenced, not reproved). Exploit names, dates, and figures are taken from public audit sources as recorded in the author’s research notes.

1. MOTIVATION: LOCAL CORRECTNESS IS NOT COMPOSITIONAL CORRECTNESS

The dominant failure mode in audited smart-contract systems is not a broken component. It is correct components that break *each other*. Trail of Bits, describing the Provenance Blockchain bug discussed below, put it in exactly these words: “two independently-correct components . . . created a security failure through incorrect interaction.”

This is the laxator-as-obstruction picture. When we compose two effectful structures S and T — two state monads, two ledgers, two order-of-effect disciplines — the composite $S \circ T$ is a well-behaved structure again *only if* a distributive law

$$\delta : ST \Longrightarrow TS$$

exists and satisfies its coherence conditions. The distributive law is exactly the datum that lets the two structures interleave without contradiction. A missing or non-coherent δ is not a cosmetic gap: it is the obstruction, and downstream it is the bug. Local correctness of S and of T says nothing about whether δ exists.

We claim the recurring audit patterns are indexed by *which* law fails, and this indexing is the contribution — the audit literature carries all three as separate “patterns” with no unifying frame:

- (1) **State divergence** — the distributive law between two component monads fails; two representations of one logical value drift apart under composition.
- (2) **Temporal / cross-callback misalignment** — classical re-entrancy; the order-of-effects obstruction living in a second cohomology group.
- (3) **Self-composition via iteration** — one locally-correct operation composed with itself in a loop; an interchange/associativity failure of the iterate.

2. THE WORKED TOY EXAMPLE: STATE DIVERGENCE AS A FAILED DISTRIBUTIVE LAW

The bug (Provenance Blockchain, Trail of Bits, 2026-08-25). Two modules each hold a notion of token supply. The *Marker* module carries a cached `supply` field; the *Bank* module carries the live, authoritative supply. Each module is correct on its own terms. An authorization check compares the two supplies to decide whether a privileged action is permitted. When the cached and the live values both read as 0 — which the composed system allows — the check degenerates to `0 == 0 → true`, granting unauthorized admin/mint/withdraw. Two correct components, one trivial bypass under composition.

The categorical model. Let L be the logical value “token supply.” Two components each present a monadic view of an L -valued state:

$$S = (-) \times L_{\text{cache}} \quad (\text{Marker's stale view}), \quad T = (-) \times L_{\text{live}} \quad (\text{Bank's authoritative view}),$$

each a writer/state monad over its own copy of L . Composing the two modules means forming $S \circ T$ and reading a single logical supply out of it. Coherence — “the two views agree on the one value they both denote” — is exactly a distributive law

$$\delta : ST \Longrightarrow TS$$

whose unit-square commutes: the value read through the cache equals the value read through the authoritative store. The intended invariant $L_{\text{cache}} = L_{\text{live}}$ is the statement that δ is well-defined and coherent.

The bug is that no such coherent δ exists. The cache and the store are updated on different code paths, so the square

$$L_{\text{cache}} \xrightarrow{\text{compose}} L_{\text{live}}$$

fails to commute after certain interleavings, and the two representations diverge. The authorization predicate $\varphi(L_{\text{cache}}, L_{\text{live}})$ was written assuming coherence; under divergence it is evaluated off its intended domain, where $L_{\text{cache}} = L_{\text{live}} = 0$ makes it fire spuriously. The divergence *is* the failed distributive law; the `0 == 0` bypass is what the missing δ looks like at the call site.

Remark 1. The model is deliberately light. What a category theorist should see is a non-existent distributive law between two monads over a shared object L ; what a security engineer should see is their own bug — “two sources of truth for one value, kept in sync by convention rather than by construction.” The two readings are the same statement.

3. THE TAXONOMY: THREE FAILURE SHAPES, THREE LAWS

(1) State divergence = failed distributive law between two monads. The shape of Section 2, and the best-populated: four public witnesses across three sub-families, each a pair of locally-correct components whose composite admits no coherent distributive law between two representations of one logical value. The sub-families are distinguished by *what* the two representations are — two modules, two entry points, or two channels — but the missing δ is the same in each.

(1a) *Module/module*. Two distinct modules each carry a view of the shared value. The Provenance toy of Section 2 is the clean case; the same shape drives the Uniswap v4 hook accounting bug in which the `PoolManager`’s delta-zero settlement check and a hook’s internal ledger are each locally sound but jointly unsound — the **Bunni exploit**, \$8.4M. Two ledgers, no distributive law between them.

(1b) *Entry-point/entry-point*. The two representations need not live in two modules: they can be two entry points into *one* component. In the **Solv Protocol BRO Vault** exploit (2026-03, ≈\$2.7M), an ERC-3525 semi-fungible vault runs an external deposit-mint (`onERC721Received`) and an internal value-rebalance (`ERC3525TransferHelper.doTransfer`) as two operations on the same underlying state (`mintedSupply` versus `collateralValue`). Both funnel through the same callback with no processed-flag guard, so rebalance-then-mint interleaves with deposit-then-mint and the

composite is ill-defined; the attacker looped `transfer`→`mint` 22 times, inflating 135 BRO into $\approx 567M$ BRO. Here the second “operation” is an internal helper re-entry, not an external attacker-contract callback — so the missing δ lives between two entry points into one component, widening the shape from “two modules” to “two entry points.”

(1c) *Channel/channel*. Sharper still: the two representations are two *channels* onto one pot. In the **Venus Protocol donation attack** (2026-03, $\approx \$14.9M$), a Compound-fork lending market reads its exchange rate from `getCashPrior()` — the raw ERC20 `balanceOf` of the `vToken` contract — rather than from an internally-tracked mint ledger. The attacker bypassed `mint()` entirely and `transfer`-ed tokens straight into the contract: the cash balance rose but the minted-share `totalSupply` did not, inflating the exchange rate $3.81\times$ and slipping past a supply cap that is checked only against mint-pathway volume. Looping `borrow`→`swap`→`donate`→`borrow` extracted $\$14.9M$ and left $\$2.18M$ of bad debt. The two non-commuting channels are a public, permissionless ERC20 transfer and the protocol’s own accounting channel onto the same pot; the distributive law is assumed to hold only because everyone is expected to route through `mint()`, and nothing enforces it. This is the general shape of ERC-4626-style “donation”/share-inflation attacks. Tellingly, the canonical fix — read a state variable mutated only at protocol entry points instead of the live balance — forces the two channels to coincide *by construction*: it literally makes the distributive law hold.

(2) Temporal / cross-callback misalignment = re-entrancy with an H^2 obstruction. Classical re-entrancy is an order-of-effects failure of a single lax structure composed across a callback boundary: an external call re-enters before the caller’s effects have settled, so the intended interleaving order is violated. This is the case the author has already pinned down formally. The obstruction is a cocycle $[\omega] \in H^2$, and the composition is safe exactly when $[\omega] = \varepsilon$, the coboundary/identity class; this equivalence is machine-checked (Reentrancy.lean, $[\omega] = \varepsilon$, with $H^2 \cong \mathbb{F}_2$). The Uniswap v4 hook patterns of temporal state misalignment across `beforeSwap/afterSwap`, and of cross-callback shared-state mutation across pools, are instances of this same failed distributive law of order.

(3) Self-composition via iteration = interchange/associativity failure. The subtlest shape: a *single* locally-correct operation composed with *itself* through a loop. In the Slither issue #940 (Hash-masks) pattern, `mintNFT` loops over `_safeMint`; each iteration individually respects checks-effects-interactions, but a reentrant call mid-loop bypasses the `MAX_NFT_SUPPLY` check that only fires once at function entry. Structurally the obstruction lives in the iterate $T \circ T \circ \dots$ of one operation, not between two distinct operations. This is the security-domain shadow of a phenomenon already visible in the author’s wreath/plethysm work: self-composition $M \circ M$ can carry a symmetry (a stabilizer / block-swap) that no single application exhibits, and composition survives self-plethysm only when flatness was present to begin with. A check that holds per step but not under the fold is a locally-flat operation whose iterate is non-flat.

Failure shape	Law that breaks	Cited exploit
State divergence (3 sub-families: module, entry-point, channel)	distributive law between two monads	Provenance Blockchain (ToB, 2026-08-25); Bunni (Uniswap v4 hook), $\$8.4M$; Solv BRO Vault, $\$2.7M$; Venus donation, $\$14.9M$
Temporal / cross-callback	order-of-effects law, $[\omega] \in H^2$; safe iff $[\omega] = \varepsilon$	Uniswap v4 hooks (<code>beforeSwap/afterSwap</code> ; cross-pool)
Self-composition (iterate)	interchange/associativity of $T \circ T \circ \dots$	Slither #940 (Hash-masks) <code>mintNFT</code> loop

Remark 2 (What is and is not claimed). This note is a framing, graded as expository. No new theorem is proved here. The one formally verified result invoked is the pre-existing re-entrancy obstruction ($[\omega] = \varepsilon$, machine-checked). The categorical models of shapes (1) and (3) are structural descriptions, not proofs. Related dynamical-systems “under-generation-under-composition” analogies exist in the literature but use no category theory and are cited elsewhere only as analogy.

4. WHY THIS MATTERS FOR THE GRANT

Compositional correctness has direct economic consequences: the Bunni exploit alone is a \$8.4M figure attached to a single missing distributive law between two ledgers, and the Venus donation attack a \$14.9M figure attached to the same failure between two channels onto one pot. The audit industry already *discovers* these bugs one at a time; what it lacks is a frame that says why they recur and where to look next.

The distributive-law frame supplies three things the grant narrative wants:

- **A unification.** Three ostensibly unrelated exploit classes are three instances of one phenomenon — composition failing for lack of a coherence datum — distinguished only by *where* the datum should live (between two monads; in the order of one lax structure; in the iterate of one operation).
- **Prediction.** The frame tells an auditor what to hunt for: two sources of truth for one value — two modules, two entry points, or two channels onto one pot (look for the distributive law); a callback boundary (look for the H^2 class); a loop over an externally-observable operation (look for the non-flat iterate). The taxonomy turns ad hoc pattern-matching into a checklist indexed by categorical structure.
- **A checkable criterion.** For the temporal case the criterion is already sharp and machine-checked: the composition is safe if and only if $[\omega] = \varepsilon$. This is the template for the other two shapes — a compositional-correctness certificate that a verification tool could in principle discharge, rather than a code smell a human hopes to notice.

In short: local correctness is cheap and compositional correctness is expensive precisely because the latter is a statement about a distributive law, not about any component. The framework names that law, predicts its absence, and — in the case already formalized — decides it.

KODAMAI

Email address: `neil@kodamai.com`